

전국기능경기대회 채점기준

1. 채점 시 유의사항

직 종 명

클라우드컴퓨팅

※ 다음 사항을 유의하여 채점하시오.

- 1) AWS의 지역은 **ap-northeast-2**을 사용합니다.
- 2) 웹페이지 접근은 크롬이나 파이어폭스를 이용합니다.
- 3) 웹페이지에서 언어에 따라 문구가 다르게 보일 수 있습니다.
- 4) **shell**에서의 명령어의 출력은 버전에 따라 조금 다를 수 있습니다.
- 5) 채점은 문항 순서대로 진행해야 합니다.
- 6) 부분 점수가 따로 없는 문항은 모두 맞아야 점수로 인정됩니다.
- 7) 리소스의 정보를 읽어오는 채점항목은 기본적으로 스크립트 결과를 통해 채점을 진행하며, 만약 선수가 이의가 있다면 명령어를 직접 입력하여 확인해볼 수 있습니다.
- 8) (예상 출력)은 바로 이전 (명령어 입력)의 예상 출력을 의미합니다.
- 9) 채점 시에는 별도로 제공한 채점 스크립트(**mark.sh**)를 실행하여 채점할 수 있습니다.
다만, 선수가 직접 입력을 원할 경우 채점기준표에 명시된 명령어 그대로 입력하여 채점할 수 있습니다.
- 10) 제출 전 **mark-sg**의 경우 선수 임의로 구성해두도록 합니다.
- 11) 모든 채점 사항은 **Cloudshell**에 **wsc2026-skills-app-sub-a** 서버넷과 사전구성한 **mark-sg**를 연결한 후 진행합니다.
- 12) 아래 명령어를 사전입력합니다.

```
aws configure set default.region ap-northeast-2
```

```
ACCOUNT_ID=$(aws sts get-caller-identity --query "Account" --output text)
```

```
VPC_ID=$(aws ec2 describe-vpcs --filters "Name=tag:Name,Values=wsc2026-skills-vpc" --query "Vpcs[0].VpcId" --output text)
```

```
BUCKET=$(aws s3api list-buckets --query "Buckets[?contains(Name,'wsc2026-static')].Name" --output text)
```

```
aws eks update-kubeconfig --name wsc2026-eks-cluster --region ap-northeast-2 2>/dev/null
```

```
check_kms() {
```

```
    local ALIAS=$1 ACTUAL_ARN=$2; local EXPECTED_ARN=$(aws kms describe-key --key-id "alias/${ALIAS}" --query "KeyMetadata.Arn" --output text 2>/dev/null); local KEY_ID=$(aws kms describe-key --key-id "alias/${ALIAS}" --query "KeyMetadata.KeyId" --output text 2>/dev/null)
    if [ -z "$KEY_ID" ] || [ "$KEY_ID" = "None" ]; then echo "KMS ${ALIAS}: FAIL (key not found)"; return; fi
```

```
    if [ "$EXPECTED_ARN" != "$ACTUAL_ARN" ]; then echo "KMS ${ALIAS}: FAIL (wrong key)"; return; fi
```

```
    POLICY=$(aws kms get-key-policy --key-id "$KEY_ID" --policy-name default --output text 2>/dev/null)
```

```
    if echo "$POLICY" | grep -q '"kms:\*"' ; then echo "KMS ${ALIAS}: FAIL (kms:*)"; elif echo "$POLICY" | grep -q ':root' ; then echo "KMS ${ALIAS}: FAIL (root)"; else echo "KMS ${ALIAS}: PASS"; fi
```

```
}
```

※ 채점기준 양식은 반드시, 양식에 맞추어 작성해야 합니다.(채점사이트 입력에 필요)

2. 채점기준표

1) 주요항목별 배점			직 종 명		클라우드컴퓨팅			
과제 번호	일련 번호	주요항목	배점	채점방법		채점시기		비고
				독립	합의	경기 진행중	경기 종료후	
제1과제	1	Networking	2.0	○			○	
	2	Database	1.3	○			○	
	3	Container Registry	1.2	○			○	
	4	Container Orchestration	3.0	○			○	
	5	Deployment	6.5	○			○	
	6	S3	1.0	○			○	
	7	Lambda	3.0	○			○	
	8	ALB	1.3	○			○	
	9	CloudFront	4.0	○			○	
	10	WAF	1.5	○			○	
	11	Observability	5.2	○			○	
합 계			30					

2) 채점방법 및 기준

과제 번호	일 련 번 호	주요항목	일련 번호	세부항목 (채점방법)	배점
제1과제	1	Networking	1	VPC	1.0
			2	Routing	1.0
	2	Database	1	DynamoDB Table	1.3
	3	Container Registry	1	ECR Container	1.2
	4	Container Orchestration	1	EKS Cluster	1.5
			2	NodeGroup	1.0
			3	EKS IAM	0.5
	5	Deployment	1	Resource Status	1.5
			2	Scheduling & Resources	1.0
			3	Probes & ConfigMap	1.0
			4	Service & Pod Placement	1.5
			5	Pod Identity	1.5
	6	S3	1	S3 Bucket	1.0
	7	Lambda	1	Lambda Functions	1.5
			2	Lambda IAM	1.5
	8	Load Balancer	1	Load Balancer	1.3
	9	CloudFront	1	CloudFront Distribution	1.2
			2	Cache Policy	1.3
			3	POST & GET E2E	1.5
	10	WAF	1	WAF Protection	1.5
	11	Observability	1	Observability Deploy	1.2
			2	Grafana Datasource	1.0
			3	Grafana Dashboard	1.5
			4	Alert Firing	1.5
합계					30

순번	채점 항목
1-1	1) <code>aws ec2 describe-vpcs --vpc-ids "\${VPC_ID}" --query "Vpcs[0].CidrBlock" --output text;</code> <code>aws ec2 describe-subnets --filters "Name=vpc-id,Values=\${VPC_ID}" --query "Subnets[0].{Name:Tags[?Key='Name'].Value [0],Cidr:CidrBlock}" --output text</code> 2) VPC와 서브넷의 이름이 정확하게 일치하며 VPC와 서브넷에 지정한 CIDR가 부여되어있는지 확인합니다. 192.168.0.0/16 192.168.1.0/24 wsc2026-skills-hub-sub-a 192.168.10.0/24 wsc2026-skills-hub-sub-b 192.168.2.0/24 wsc2026-skills-app-sub-a 192.168.20.0/24 wsc2026-skills-app-sub-b
1-2	1) <code>IGW_ID=\$(aws ec2 describe-internet-gateways --filters "Name=attachment.vpc-id,Values=\${VPC_ID}" "Name=tag:Name,Values=wsc2026-skills-igw" --query "InternetGateways[0].InternetGatewayId" --output text); echo "IGW: \$IGW_ID"</code> <code>NAT_A_ID=\$(aws ec2 describe-nat-gateways --filter "Name=vpc-id,Values=\${VPC_ID}" "Name=tag:Name,Values=wsc2026-skills-nat-a" "Name=state,Values=available" --query "NatGateways[0].NatGatewayId" --output text);</code> <code>NAT_B_ID=\$(aws ec2 describe-nat-gateways --filter "Name=vpc-id,Values=\${VPC_ID}" "Name=tag:Name,Values=wsc2026-skills-nat-b" "Name=state,Values=available" --query "NatGateways[0].NatGatewayId" --output text); echo "NAT-A: \$NAT_A_ID"; echo "NAT-B: \$NAT_B_ID"</code> <code>aws ec2 describe-route-tables --filters "Name=vpc-id,Values=\${VPC_ID}" "Name=tag:Name,Values=*" --query "RouteTables[0].{Name:Tags[?Key='Name'].Value [0],Route:Routes[?DestinationCidrBlock='0.0.0.0/0'] [0]}" --output text</code> 2) 실행 결과가 강조된 텍스트와 동일하며, hub-rtb에는 igw의 ID가, app-rtb-a에는 NAT-A의 ID가, app-rtb-b에는 NAT-B의 ID가 서로 일치하게 연결되어 있는지 확인합니다. IGW: igw-0a85cd67732857621 NAT-A: nat-071e3d4820939525d NAT-B: nat-03f60025e555f65bb wsc2026-skills-app-rtb-b ROUTE 0.0.0.0/0 nat-03f60025e555f65bb CreateRoute active wsc2026-skills-hub-rtb ROUTE 0.0.0.0/0 igw-0a85cd67732857621 CreateRoute active wsc2026-skills-app-rtb-a ROUTE 0.0.0.0/0 nat-071e3d4820939525d CreateRoute active
2-1	1) <code>aws dynamodb describe-table --table-name wsc2026-book-table --query "Table.[KeySchema[0].AttributeName,BillingModeSummary.BillingMode,SSEDescription.SSEType,DeletionProtectionEnabled,GlobalSecondaryIndexes[0].KeySchema[0].AttributeName]" --output text 2>/dev/null</code> <code>aws dynamodb describe-continuous-backups --table-name wsc2026-book-table --query "ContinuousBackupsDescription.[PointInTimeRecoveryDescription.PointInTimeRecoveryStatus,PointInTimeRecoveryDescription.RecoveryPeriodInDays,ContinuousBackupsStatus]" --output text 2>/dev/null</code> <code>aws dynamodb get-resource-policy --resource-arn</code>

순번	채점 항목
	<pre>"arn:aws:dynamodb:ap-northeast-2:\${ACCOUNT_ID}:table/wsc2026-book-table" --query "Policy" --output text 2>/dev/null python3 -c "import sys,json;[print(f' {s.get(\"Action\",\"\")} : {s.get(\"Principal\",{}).get(\"AWS\",\"\").split(\"/\")[1]}') for s in json.loads(sys.stdin.read()).get('Statement',[])]" 2>/dev/null echo "No resource policy" check_kms "wsc2026-db-kms" "\$(aws dynamodb describe-table --table-name wsc2026-book-table --query 'Table.SSEDescription.KMSMasterKeyArn' --output text 2>/dev/null)" 2) 실행 결과가 아래와 정확히 일치해야 합니다. 단, 강조된 부분의 출력 순서는 무관합니다. client_id PAY_PER_REQUEST KMS True booking_id ENABLED 35 ENABLED <u>dynamodb:PutItem : wsc2026-book-pod-role</u> <u>dynamodb:Query : wsc2026-book-function-role</u> KMS wsc2026-db-kms: PASS</pre>
3-1	<pre>1) aws ecr describe-repositories --repository-names wsc2026-book-ecr --query "repositories[0].[imageScanningConfiguration.scanOnPush,imageTagMutability,imageTagMutabil ityExclusionFilters[0].filter,encryptionConfiguration.encryptionType]" --output text 2>/dev/null; aws ecr list-images --repository-name wsc2026-book-ecr --query "imageIds[0].imageTag" --output text 2>/dev/null check_kms "wsc2026-ecr-kms" "\$(aws ecr describe-repositories --repository-names wsc2026-book-ecr --query 'repositories[0].encryptionConfiguration.kmsKey' --output text 2>/dev/null)" 2) 실행 결과가 아래와 동일한지 확인합니다. True MUTABLE_WITH_EXCLUSION v1* KMS v1.0.0 KMS wsc2026-ecr-kms: PASS</pre>
4-1	<pre>1) aws eks describe-cluster --name wsc2026-eks-cluster --query "cluster.[version,status,re sourcesVpcConfig.endpointPublicAccess,resourcesVpcConfig.endpointPrivateAccess,logging.clu sterLogging[0].enabled]" --output text 2>/dev/null CLUSTER_SG=\$(aws eks describe-cluster --name wsc2026-eks-cluster --query "cluster.resource sVpcConfig.clusterSecurityGroupId" --output text 2>/dev/null); ANYIP=\$(aws ec2 describe-se curity-groups --group-ids "\$CLUSTER_SG" --query "SecurityGroups[0].IpPermissions[?IpProtoc ol='-1'].IpRanges[0].CidrIp" --output text 2>/dev/null); if [-n "\$ANYIP"]; then echo "S G: FAIL"; else echo "SG: PASS"; fi check_kms "wsc2026-eks-kms" "\$(aws eks describe-cluster --name wsc2026-eks-cluster --query 'cluster.encryptionConfig[0].provider.keyArn' --output text 2>/dev/null)" 2) 실행 결과가 아래와 동일한지 확인합니다. 1.35 ACTIVE False True True SG: PASS wsc2026.skills.local KMS wsc2026-eks-kms: PASS</pre>
4-2	<pre>1) NG1=\$(aws eks describe-nodegroup --cluster-name wsc2026-eks-cluster --nodegroup-name ws c2026-addon-nodegroup --query "nodegroup.[nodegroupName,instanceTypes[0]]" --output text</pre>

순번	채점 항목
	<pre>2>/dev/null); echo "\$NG1 wsc2026/node=addon" ADDON_COUNT=\$(kubectl get nodes -l wsc2026/node=addon --no-headers --request-timeout=10s 2>/dev/null wc -l) if ["\$ADDON_COUNT" -ge 1]; then echo "Addon Nodes: PASS (\$ADDON_COUNT)"; else echo "Addo n Nodes: FAIL"; fi NG2=\$(aws eks describe-nodegroup --cluster-name wsc2026-eks-cluster --nodegroup-name wsc20 26-workload-ng --query "nodegroup.[nodegroupName,instanceTypes[0]]" --output text 2>/dev/n ull); echo "\$NG2 wsc2026/node=application" WORK_COUNT=\$(kubectl get nodes -l wsc2026/node=application --no-headers --request-timeout= 10s 2>/dev/null wc -l) if ["\$WORK_COUNT" -ge 1]; then echo "Workload Nodes: PASS (\$WORK_COUNT)"; else echo "Wor kload Nodes: FAIL"; fi 2) 실행 결과가 아래와 동일한지 확인합니다. wsc2026-addon-nodegroup t3.medium wsc2026/node=addon Addon Nodes: PASS (2) wsc2026-workload-ng t3.medium wsc2026/node=application Workload Nodes: PASS (2)</pre>
4-3	<pre>1) CR=\$(aws eks describe-cluster --name wsc2026-eks-cluster --query "cluster.roleArn" --ou tput text 2>/dev/null awk -F/ '{print \$NF}'); AR=\$(aws eks describe-nodegroup --cluster- name wsc2026-eks-cluster --nodegroup-name wsc2026-addon-nodegroup --query "nodegroup.nodeR ole" --output text 2>/dev/null awk -F/ '{print \$NF}'); WR=\$(aws eks describe-nodegroup -- -cluster-name wsc2026-eks-cluster --nodegroup-name wsc2026-workload-ng --query "nodegroup. nodeRole" --output text 2>/dev/null awk -F/ '{print \$NF}') for PAIR in "Cluster Role:\$CR" "Addon Node Role:\$AR" "Workload Node Role:\$WR"; do LABEL=" \${PAIR%*:}"; ROLE="\${PAIR#*:}"; ADMIN=\$(aws iam list-attached-role-policies --role-name " \$ROLE" --query "AttachedPolicies[?PolicyName=='AdministratorAccess'].PolicyName" --output text 2>/dev/null); if [-n "\$ADMIN"]; then echo "\$LABEL: FAIL"; else echo "\$LABEL: PASS"; fi; done 2) 실행 결과가 아래와 동일한지 확인합니다. Cluster Role: PASS Addon Node Role: PASS Workload Node Role: PASS</pre>
5-1	<pre>1) kubectl get deploy,svc,ingress,pdb -n wsc2026 --no-headers 2>/dev/null awk ' /^deployment/ { split(\$2,a,"/"); print "Deployment: " (a[1]==a[2] && a[1]=="2" ? "PASS" : "FAIL") } /^service/ { print "Service: PASS" } /^ingress/ { print "Ingress: " (\$4 ~ /\.elb\.amazonaws\.com/ ? "PASS (" \$4)" : "FAIL") } /^poddisrupt/ { print "PDB: " (\$2=="1" ? "PASS" : "FAIL") } ' 2) 실행결과가 아래와 동일한지 확인하며, Ingress의 ALB DNS는 wsc2026-app-alb가 포함되었는지 확인합니다.</pre>

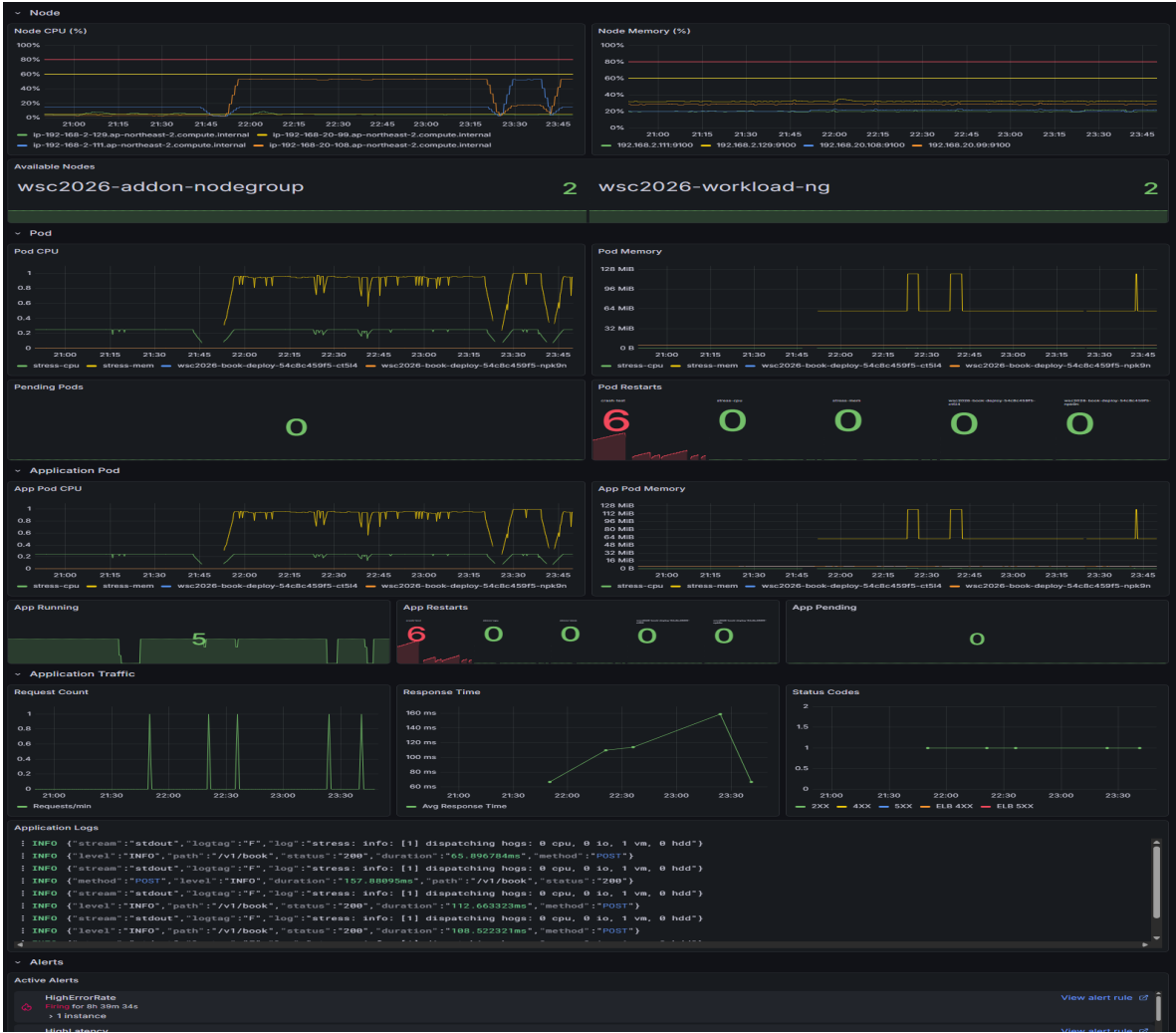
순번	채점 항목
	Deployment: PASS Service: PASS Ingress: PASS (wsc2026-app-alb-829903010.ap-northeast-2.elb.amazonaws.com) PDB: PASS
5-2	1) <code>kubectl get deploy wsc2026-book-deploy -n wsc2026 -o jsonpath='{.spec.replicas} node:{.spec.template.spec.nodeSelector.wsc2026/node} topo:{.spec.template.spec.topologySpreadConstraints[0].topologyKey} cpu:{.spec.template.spec.containers[0].resources.requests.cpu} mem:{.spec.template.spec.containers[0].resources.requests.memory}' 2>/dev/null</code> 2) 실행 결과가 아래와 동일한지 확인합니다. replicas:2 node:application topo:topology.kubernetes.io/zone cpu:250m mem:512Mi
5-3	1) <code>kubectl get deploy wsc2026-book-deploy -n wsc2026 -o jsonpath='{startup:{.spec.template.spec.containers[0].startupProbe.httpGet.path}:{.spec.template.spec.containers[0].startupProbe.httpGet.port} readiness:{.spec.template.spec.containers[0].readinessProbe.httpGet.path}:{.spec.template.spec.containers[0].readinessProbe.httpGet.port} liveness:{.spec.template.spec.containers[0].livenessProbe.httpGet.path}:{.spec.template.spec.containers[0].livenessProbe.httpGet.port}}' 2>/dev/null; echo; kubectl get configmap book-config -n wsc2026 -o jsonpath='{.data}' 2>/dev/null; echo</code> 2) 실행 결과가 아래와 동일한지 확인합니다. startup:/health:8080 readiness:/health:8080 liveness:/health:8080 {"AWS_REGION":"ap-northeast-2","TABLE_NAME":"wsc2026-book-table"}
5-4	1) <code>WORKLOAD_NODES=\$(kubectl get nodes -l wsc2026/node=application --no-headers -o custom-columns='NAME:.metadata.name' 2>/dev/null)</code> <code>kubectl get pods -n wsc2026 -o custom-columns='NAME:.metadata.name,STATUS:.status.phase,NODE:.spec.nodeName' --no-headers 2>/dev/null while read NAME STATUS NODE; do</code> <code>echo "\$NAME: \$(echo "\$WORKLOAD_NODES" grep -q "\$NODE" && echo PASS echo FAIL)"</code> <code>done</code> 2) 실행 결과에서 wsc2026-book-deploy가 포함된 텍스트가 2개 출력되며, 모두 PASS인지 확인합니다. wsc2026-book-deploy-7b88597d86-d6l72: PASS wsc2026-book-deploy-7b88597d86-z8nkt: PASS
5-5	1) <code>PI_SA=\$(aws eks list-pod-identity-associations --cluster-name wsc2026-eks-cluster --namespace wsc2026 --query 'associations[0].serviceAccount' --output text 2>/dev/null)</code> <code>PI_ROLE=\$(aws eks list-pod-identity-associations --cluster-name wsc2026-eks-cluster --namespace wsc2026 --query 'associations[0].associationId' --output text 2>/dev/null)</code> <code>["\$PI_SA" = "wsc2026-book-sa"] && echo "Pod Identity SA: PASS (\$PI_SA)" echo "Pod Identity SA: FAIL (\$PI_SA)"</code> <code>ROLE_NAME=\$(aws eks describe-pod-identity-association --cluster-name wsc2026-eks-cluster --association-id "\$PI_ROLE" --query 'association.roleArn' --output text 2>/dev/null awk -F/ '{print \$NF}')</code> <code>MANAGED_ACTIONS=\$(aws iam list-attached-role-policies --role-name "\$ROLE_NAME" --query 'AttachedPolicies[].PolicyArn' --output text 2>/dev/null tr '\t' '\n' while read arn;do</code> <code>VERSION=\$(aws iam get-policy --policy-arn "\$arn" --query 'Policy.DefaultVersionId'</code>

순번	채점 항목
	<pre>—output text 2>/dev/null);aws iam get-policy-version —policy-arn "\$arn" —version-id "\$VERSION" —query 'PolicyVersion.Document.Statement[].Action' —output text 2>/dev/null;done) echo "\$MANAGED_ACTIONS" grep -q dynamodb:PutItem && ! echo "\$MANAGED_ACTIONS" grep -qE '^*\$' && echo "Pod Identity Role: PASS" echo "Pod Identity Role: FAIL"</pre> 2) 실행 결과가 아래와 동일한지 확인합니다. Pod Identity SA: PASS (wsc2026-book-sa) Pod Identity Role: PASS
6-1	<pre>1) echo "\$BUCKET"; aws s3api get-public-access-block —bucket "\$BUCKET" —query "PublicAccessBlockConfiguration.[BlockPublicAcls,BlockPublicPolicy,IgnorePublicAcls,RestrictPublicBuckets]" —output text 2>/dev/null; aws s3api get-bucket-encryption —bucket "\$BUCKET" —query "ServerSideEncryptionConfiguration.Rules[0].{SSE:ApplyServerSideEncryptionByDefault.SSEAlgorithm,BucketKey:BucketKeyEnabled}" —output text 2>/dev/null; aws s3api list-objects —bucket "\$BUCKET" —prefix "static/" —query "Contents[?Size>\0\].Key" —output text 2>/dev/null check_kms "wsc2026-bucket-kms" "\$(aws s3api get-bucket-encryption —bucket "\$BUCKET" —query 'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault.KMSMasterKeyID' —output text 2>/dev/null)" BUCKET_KMS_ARN=\$(aws kms describe-key —key-id "alias/wsc2026-bucket-kms" —query "KeyMetadata.Arn" —output text 2>/dev/null) echo "S3 Object KMS Check:"; for OBJ in \$(aws s3api list-objects —bucket "\$BUCKET" —prefix "static/" —query "Contents[].Key" —output text 2>/dev/null); do KEY_ID=\$(aws s3api head-object —bucket "\$BUCKET" —key "\$OBJ" —query "SSEKMSKeyId" —output text 2>/dev/null) if ["\$KEY_ID" = "\$BUCKET_KMS_ARN"]; then echo " \$OBJ: PASS"; else echo " \$OBJ: FAIL (\$KEY_ID)"; fi done</pre> 2) 실행 결과가 아래와 동일한지 확인합니다. wsc2026-static<임의 영문 4자리><선수 비번호>-bucket <pre>True True True True True aws:kms static/index.html static/main.jpeg KMS wsc2026-bucket-kms: PASS S3 Object KMS Check: static/: PASS static/index.html: PASS static/main.jpeg: PASS</pre>
7-1	<pre>1) aws lambda get-function —function-name wsc2026-book-get-function —query "Configuration.{Runtime:Runtime,Env:Environment.Variables.TABLE_NAME}" —output text 2>/dev/null check_kms "wsc2026-function-kms" "\$(aws lambda get-function —function-name wsc2026-book-get-function —query 'Configuration.KMSKeyArn' —output text 2>/dev/null)"</pre> 2) 실행 결과가 아래와 동일한지 확인하며, TABLE_NAME은 평문이 아닌 KMS로 암호화된 텍스트가 출력되는지 확인합니다. <pre>{</pre>

순번	채점 항목
	<pre>"Name": "wsc2026-book-get-function", "Runtime": "python3.12" } { "TABLE_NAME": "AQICAHjHYzx8HkYI0X8fnCJbPqPanGMX7gxnAX57CFiSG1wT1wEpFZotnxm3...." } } KMS wsc2026-function-kms: PASS</pre>
7-2	1) LAMBDA_ROLE=\$(aws lambda get-function --function-name wsc2026-book-get-function --query "Configuration.Role" --output text 2>/dev/null awk -F/ '{print \$NF}'); echo "\$LAMBDA_ROLE"; POLICIES=\$(aws iam list-attached-role-policies --role-name "\$LAMBDA_ROLE" --query "AttachedPolicies[].PolicyName" --output text 2>/dev/null); echo "\$POLICIES"; if echo "\$POLICIES" grep -q "AdministratorAccess"; then echo "Role: FAIL (Admin)"; else echo "Role: PASS"; fi POLICY_ARN=\$(aws iam list-attached-role-policies --role-name "\$LAMBDA_ROLE" --query "AttachedPolicies[?PolicyName!='AWSLambdaBasicExecutionRole'].PolicyArn [0]" --output text 2>/dev/null); ACTIONS=\$(aws iam get-policy-version --policy-arn "\$POLICY_ARN" --version-id \$(aws iam get-policy --policy-arn "\$POLICY_ARN" --query "Policy.DefaultVersionId" --output text 2>/dev/null) --query "PolicyVersion.Document.Statement[].Action" --output text 2>/dev/null) if echo "\$ACTIONS" grep -q "dynamodb:Query" && ! echo "\$ACTIONS" grep -q '*'; then echo "Policy: PASS"; else echo "Policy: FAIL"; fi 2) 실행 결과가 아래와 동일한지 확인합니다. <pre>wsc2026-book-function-role wsc2026-book-function-policy Role: PASS Policy: PASS</pre>
8-1	1) aws elbv2 describe-load-balancers --names wsc2026-app-alb --query "LoadBalancers[0].Scheme" --output text 2>/dev/null; ALB_SGS=\$(aws elbv2 describe-load-balancers --names wsc2026-app-alb --query "LoadBalancers[0].SecurityGroups[]" --output text 2>/dev/null); aws ec2 describe-security-groups --group-ids \$ALB_SGS --query "SecurityGroups[].GroupName" --output text 2>/dev/null ALB_DNS=\$(aws elbv2 describe-load-balancers --names wsc2026-app-alb --query "LoadBalancers[0].DNSName" --output text 2>/dev/null); ALB_CODE=\$(curl -s -o /dev/null -w "%{http_code}" --max-time 5 "http://\${ALB_DNS}/" 2>/dev/null); if ["\$ALB_CODE" = "000"]; then echo "ALB direct: BLOCKED"; else echo "ALB direct: FAIL (\$ALB_CODE)"; fi 2) 실행 결과가 아래와 동일한지 확인합니다. <pre>internet-facing wsc2026-app-alb-sg ALB direct: BLOCKED</pre>
9-1	1) CF_ARN=\$(aws resourcegroupstaggingapi get-resources --tag-filters Key=Name,Values=wsc2026-cdn --resource-type-filters cloudfront --region us-east-1 --query "ResourceTagMappingList[0].ResourceARN" --output text 2>/dev/null); CF_ID=\$(echo "\$CF_ARN"

순번	채점 항목
	<pre> awk -F/ '{print \$NF}'); CF_DOMAIN=\$(aws cloudfront get-distribution --id "\$CF_ID" --region us-east-1 --query "Distribution.DomainName" --output text 2>/dev/null); curl -s -o /dev/null -w "\${CF_DOMAIN} : %{http_code}" "https://\${CF_DOMAIN}/" 2>/dev/null; echo</pre> 2) 실행 결과에서 Cloudfront 도메인과 200이 출력되는지 확인합니다. d1ain3jwa2akkh.cloudfront.net : 200
9-2	<pre>1) DISABLED="4135ea2d-6df8-44a3-9df3-4b5a84be39ad"; OPTIMIZED="658327ea-f89d-4fab-a63d-7e88639e58f6"; DEFAULT_CP=\$(aws cloudfront get-distribution --id "\$CF_ID" --region us-east-1 --query "Distribution.DistributionConfig.DefaultCacheBehavior.CachePolicyId" --output text 2>/dev/null); if ["\$DEFAULT_CP" = "\$OPTIMIZED"]; then echo "S3: CachingOptimized"; else echo "S3: FAIL"; fi</pre> <pre>for B in \$(aws cloudfront get-distribution --id "\$CF_ID" --region us-east-1 --query "Distribution.DistributionConfig.CacheBehaviors.Items[].PathPattern" --output text 2>/dev/null); do CP=\$(aws cloudfront get-distribution --id "\$CF_ID" --region us-east-1 --query "Distribution.DistributionConfig.CacheBehaviors.Items[?PathPattern='\$B'].CachePolicyId [0]" --output text 2>/dev/null); if ["\$CP" = "\$DISABLED"]; then echo "ALB/Lambda: CachingDisabled"; break; fi; done</pre> 2) 실행 결과가 아래와 동일한지 확인합니다. S3: CachingOptimized ALB/Lambda: CachingDisabled
9-3	<pre>1) BOOKING_ID=\$(curl -s -X POST "https://\${CF_DOMAIN}/booking" -H "Content-Type: application/json" -d '{"client_id":"MARK001","username":"Marker","email":"mark@test.com","concert_name":"TestConcert"}' 2>/dev/null python3 -c "import sys,json;print(json.load(sys.stdin).get('booking_id',''))" 2>/dev/null); echo "POST booking_id: \$BOOKING_ID"</pre> <pre>curl -s "https://\${CF_DOMAIN}/v1/book?booking_id=\$BOOKING_ID" 2>/dev/null; echo</pre> 2) 필드 순서(client_id, username, email, concert_name, created_at)가 차례대로 일치하는지, 그리고 created_at 시간이 스크립트 실행 시간 기준 1분 이내인지 확인합니다. POST booking_id: NWOEMIU2 <pre>{"client_id": "MARK001", "username": "Marker", "email": "mark@test.com", "concert_name": "TestConcert", "created_at": "2026-05-15 00:13:05 KST"}</pre>
10-1	<pre>1) WAF_NAME=\$(aws wafv2 list-web-acls --scope CLOUDFRONT --region us-east-1 --query "WebACLs[?contains(Name, 'wsc2026')].Name" --output text 2>/dev/null)</pre> <pre>echo "WAF Name: \$WAF_NAME"</pre> <pre>SQLI=\$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 "https://\${CF_DOMAIN}/v1/book?booking_id=1'%20OR%201=1--"); echo "SQLi: \$SQLI"</pre> <pre>XSS=\$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 "https://\${CF_DOMAIN}/v1/book?booking_id=<script>alert(1)</script>"); echo "XSS: \$XSS"</pre> <pre>(for i in \$(seq 1 15); do for j in \$(seq 1 25); do curl -s -o /dev/null --max-time 3 "https://\${CF_DOMAIN}/" & done; sleep 0.4; done; wait) >/dev/null 2>&1; WAF_ID=\$(aws wafv2 list-web-acls --scope CLOUDFRONT --region us-east-1 --query "WebACLs[?contains(Name, 'wsc2026')].Id" --output text 2>/dev/null); RATE_LIMIT=\$(aws wafv2 get-web-acl --scope CLOUDFRONT --region us-east-1 --name "\$WAF_NAME" --id "\$WAF_ID" --query "WebACL.Rules[?Statement.RateBasedStatement].Statement.RateBasedStatement.Limit" --output text 2>/dev/null); if [-n "\$RATE_LIMIT"] && ["\$RATE_LIMIT" -le 200] 2>/dev/null; then echo "Rate: PASS (403)"; else echo "Rate: FAIL (-)"; fi</pre>

순번	채점 항목
	2) 실행 결과가 아래와 동일한지 확인합니다. WAF Name: wsc2026-waf SQLi: 403 XSS: 403 Rate: PASS (403)
11-1	1) SVC_IP=\$(kubectl get svc -n wsc2026 -o jsonpath='{.items[0].spec.clusterIP}' 2>/dev/null); kubectl run not-ready --image=busybox --restart=Always -n wsc2026 --overrides='{\"spec\":{\"tolerations\": [{\"operator\":\"Exists\"}],\"nodeSelector\":{\"wsc2026/node\":\"application\"},\"containers\": [{\"name\":\"not-ready\", \"image\":\"busybox\", \"readinessProbe\":{\"httpGet\":{\"path\":\"/health\", \"port\":80}, \"periodSeconds\":3}, \"command\": [\"sh\", \"-c\", \"sleep 3600\"]}]}}' &>/dev/null; kubectl run error-gen --image=curlimages/curl --restart=Never -n wsc2026 --overrides='{\"spec\":{\"tolerations\": [{\"operator\":\"Exists\"}],\"nodeSelector\":{\"wsc2026/node\":\"application\"}}}' -- sh -c 'while true; do curl -s -o /dev/null http://\"\$SVC_IP\"/nonexist; sleep 0.1; done' &>/dev/null; kubectl run latency-gen --image=curlimages/curl --restart=Never -n wsc2026 --overrides='{\"spec\":{\"tolerations\": [{\"operator\":\"Exists\"}],\"nodeSelector\":{\"wsc2026/node\":\"application\"}}}' -- sh -c 'while true; do curl -s -o /dev/null http://\"\$SVC_IP\"/delay?ms=5000; sleep 0.2; done' &>/dev/null kubectl run crash-test --image=busybox --restart=Always -n wsc2026 --overrides='{\"spec\":{\"tolerations\": [{\"operator\":\"Exists\"}],\"nodeSelector\":{\"wsc2026/node\":\"application\"}}}' -- sh -c 'exit 1' &>/dev/null; kubectl run stress-cpu --image=busybox --restart=Never -n wsc2026 --overrides='{\"spec\":{\"tolerations\": [{\"operator\":\"Exists\"}],\"nodeSelector\":{\"wsc2026/node\":\"application\"},\"containers\": [{\"name\":\"stress-cpu\", \"image\":\"busybox\", \"resources\": {\"requests\": {\"cpu\":\"250m\"}, \"limits\": {\"cpu\":\"250m\"}}, \"command\": [\"sh\", \"-c\", \"while true; do :; done\"]}]}}' &>/dev/null; kubectl run stress-mem --image=polinux/stress --restart=Never -n wsc2026 --overrides='{\"spec\":{\"tolerations\": [{\"operator\":\"Exists\"}],\"nodeSelector\":{\"wsc2026/node\":\"application\"},\"containers\": [{\"name\":\"stress-mem\", \"image\":\"polinux/stress\", \"resources\": {\"requests\": {\"memory\":\"64Mi\"}, \"limits\": {\"memory\":\"64Mi\"}}, \"command\": [\"stress\", \"-vm\", \"1\", \"-vm-bytes\", \"60M\", \"-vm-keep\", \"-t\", \"3600\"]}]}}' &>/dev/null sleep 180 GRAFANA_LB=\$(kubectl get svc -n observability -o jsonpath='{range .items[?(@.spec.type==\"LoadBalancer\")]}{.status.loadBalancer.ingress[0].hostname}{end}' 2>/dev/null) for p in fluent-bit prometheus grafana; do kubectl get pods -n observability --no-headers --request-timeout=10s 2>/dev/null grep -c \"\$p.*Running\" xargs -I{} echo \"\$p: {}\"; done 2) 실행 결과가 1 이상인지 확인합니다. fluent-bit: 4 prometheus: 7 grafana: 1
11-2	1) echo \"Datasources:\"; curl -s -u admin:'Skills\$\$@!' \"http://\${GRAFANA_LB}/api/datasources\" 2>/dev/null python3 -c \"import sys,json;[print(f' {d[\\\"name\\\"]} ({d[\\\"type\\\"]})') for d in json.load(sys.stdin)]\" 2>/dev/null echo \" Grafana unreachable\" echo \"Dashboards:\"; curl -s -u admin:'Skills\$\$@!' \"http://\${GRAFANA_LB}/api/search?query=wsc2026\" 2>/dev/null python3 -c \"import sys,json;[print(f' {d[\\\"title\\\"]}') for d in json.load(sys.stdin)]\" 2>/dev/null echo \" Not found\" 2) 실행 결과가 아래와 동일한지 확인합니다. Datasources: alertmanager (alertmanager)

순번	채점 항목
	cloudwatch (cloudwatch) prometheus (prometheus) Dashboards: wsc2026-grafana-dashboard
11-3	1) 출력된 Grafana URL을 통해 admin/Skills\$\$@!에 로그인 후 wsc2026-grafana-dashboard에서 
11-4	1) Active Alerts 패널에 Edit을 선택해 나온 결과와 채점지 사진과 일치하는지 확인합니다.

순번	채점 항목
	Active Alerts HighErrorRate Firing for 2h 14m 57s > 1 instance View alert rule HighLatency Firing for 2h 14m 57s > 1 instance View alert rule PodCrashLooping Firing for 4m 49s > 1 instance View alert rule PodHighCPU Firing for 2m 49s > 1 instance View alert rule PodHighMemory Firing for 6m 49s > 1 instance View alert rule PodNotReady Firing for 6m 49s > 1 instance View alert rule